

Relaciones y Funciones

Matemática Discreta — MM420

Capítulo 5

Universidad Nacional Autónoma de Honduras
Facultad de Ciencias
Departamento de Matemáticas

II Período, 2026

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Pregunta central del capítulo

Hasta ahora hemos estudiado conjuntos como colecciones de objetos. *¿Cómo describimos matemáticamente la **interacción** entre elementos de uno o varios conjuntos?*

- Un alumno *pertenece* a una sección.
- Un número *divide* a otro.
- Una función $f(x) = x^2$ *asigna* a cada x su cuadrado.

Todas estas afirmaciones son **relaciones** o **funciones**. Este capítulo construye el lenguaje formal para manejarlas.

El capítulo sigue una **escalera de generalización**:

- 1 Construimos el *escenario*: **productos cartesianos**.
- 2 Sobre ellos definimos **relaciones** y, en particular, **funciones**.
- 3 Clasificamos las funciones (inyectivas, sobreyectivas, biyectivas).
- 4 Las **combinamos** (composición) y las **deshacemos** (inversa).
- 5 Aplicamos todo en **conteo**, en el **principio del palomar** y en **complejidad computacional**.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 **Productos cartesianos y relaciones**
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Definición

Sean A y B conjuntos. El **producto cartesiano** $A \times B$ es el conjunto de todos los **pares ordenados** (a, b) con $a \in A$ y $b \in B$:

$$A \times B = \{(a, b) \mid a \in A \text{ y } b \in B\}.$$

Tamaño

Si A y B son finitos, $|A \times B| = |A| \cdot |B|$.

- El par es **ordenado**: $(a, b) \neq (b, a)$ en general.
- Se extiende a n conjuntos: $A_1 \times \cdots \times A_n$.

Definición

Una **relación** $\mathcal{R}$ de A en B es cualquier subconjunto de $A \times B$. Cuando $A = B$, hablamos de una **relación en** A .

Notación

En lugar de $(a, b) \in \mathcal{R}$ escribimos $a \mathcal{R} b$.

Ejemplos que ya conoce el lector

- “ $\leq$ ” en $\mathbb{Z}$ es una relación en $\mathbb{Z}$.
- “divide a” en $\mathbb{N}$: $a \mid b$.
- “está matriculado en” entre alumnos y secciones.

¿Por qué nos interesan?

Toda interacción entre conjuntos es una relación. A continuación veremos el caso particular más importante: las **funciones**.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general**
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Función: definición

Definición

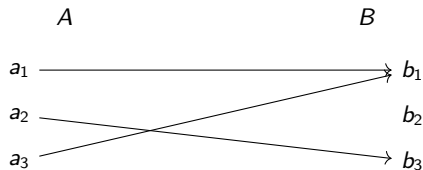
Una **función** f de A en B , escrita $f : A \rightarrow B$, es una relación de A en B en la que **cada elemento de A aparece como primera componente de exactamente un par.**

- A es el **dominio** de f ; B es el **codominio**.
- Si $f(a) = b$: b es la **imagen** de a , y a es una **preimagen** de b .
- El **rango** de f es $f(A) = \{f(a) \mid a \in A\} \subseteq B$.

Idea clave

Una función es una **regla de correspondencia** sin ambigüedad: cada $a \in A$ tiene *una y solo una* imagen.

Funciones como diagramas



- Cada elemento de A tiene **exactamente una** flecha saliendo.
- Algunos elementos de B pueden no recibir flechas (no es sobreyectiva).
- Algunos pueden recibir varias flechas (no es inyectiva).

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas**
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Definición

$f : A \rightarrow B$ es **inyectiva** (o *uno a uno*) si elementos distintos de A van a elementos distintos de B :

$$\forall a_1, a_2 \in A, f(a_1) = f(a_2) \implies a_1 = a_2.$$

- Equivale a: $a_1 \neq a_2 \implies f(a_1) \neq f(a_2)$.
- Si A es finito y f es inyectiva, entonces $|A| \leq |B|$.

Idea visual

Las flechas de la función **nunca se juntan** en el codominio.

Definición

$f : A \rightarrow B$ es **sobreyectiva** (o *subyectiva*) si todo elemento de B es imagen de algún elemento de A :

$$\forall b \in B, \exists a \in A \text{ con } f(a) = b.$$

Es decir, $f(A) = B$.

- Si A y B son finitos y f es sobreyectiva, entonces $|A| \geq |B|$.
- **Biyectiva** = inyectiva y sobreyectiva.

Idea visual

Las flechas **cubren todo** el codominio: cada $b \in B$ recibe al menos una.

Las tres clases, en una mirada

Propiedad	Inyectiva	Sobreyectiva	Biyectiva
Flechas distintas en A <i>llegan</i> a distintos b ?	Sí	—	Sí
Toda $b \in B$ recibe alguna flecha?	—	Sí	Sí
Cardinalidad (finitos)	$ A \leq B $	$ A \geq B $	$ A = B $

Una biyección entre A y B finitos es, en particular, un **emparejamiento perfecto**: los dos conjuntos tienen el mismo tamaño.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling**
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

El problema del conteo

Ya sabemos qué es una función sobreyectiva.

Pregunta de conteo

¿Cuántas funciones sobreyectivas $f : A \rightarrow B$ existen si $|A| = n$ y $|B| = k$ con $n \geq k$?

La respuesta requiere una idea intermedia: **particionar** A en k bloques no vacíos, uno por cada elemento de B .

Números de Stirling del segundo tipo

Definición

Para enteros $n, k \geq 0$, el **número de Stirling de segundo tipo** $S(n, k)$ es el **número de formas de particionar un conjunto de n elementos en k subconjuntos no vacíos**.

- Convención: $S(n, 0) = 0$ si $n > 0$, y $S(0, 0) = 1$.
- **Casos base:** $S(n, 1) = 1$, $S(n, n) = 1$.
- **Recurrencia:**

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1).$$

- $k \cdot S(n-1, k)$: el elemento nuevo se agrega a uno de los k bloques existentes.
- $S(n-1, k-1)$: el elemento nuevo forma un bloque aparte.

De particiones a funciones sobreyectivas

Teorema

El número de funciones sobreyectivas de un conjunto A de n elementos en un conjunto B de k elementos ($n \geq k$) es

$$k! \cdot S(n, k).$$

Idea de la demostración

- 1 Particionar A en k bloques no vacíos: $S(n, k)$ formas.
- 2 Asignar cada bloque a un elemento distinto de B : $k!$ formas.

Esto cierra el primer uso del capítulo: contar funciones vía combinatoria.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales**
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Funciones constantes, identidad, piso y techo

Función constante

$f : A \rightarrow B$ es **constante** si existe $b_0 \in B$ tal que $f(a) = b_0$ para todo $a \in A$.

Función identidad

La **identidad** en A es $\text{id}_A : A \rightarrow A$ definida por $\text{id}_A(a) = a$. Es *biyectiva* y actúa como neutro para la composición.

Función piso y techo

Para $x \in \mathbb{R}$:

$$\lfloor x \rfloor = \text{mayor entero} \leq x, \quad \lceil x \rceil = \text{menor entero} \geq x.$$

Función característica y factorial

Función característica

Para $X \subseteq U$,

$$\chi_X : U \rightarrow \{0, 1\}, \quad \chi_X(u) = \begin{cases} 1 & \text{si } u \in X, \\ 0 & \text{si } u \notin X. \end{cases}$$

Codifica subconjuntos como funciones: $\mathcal{P}(U) \leftrightarrow \{0, 1\}^U$.

Factorial y combinaciones

Para $n \geq 0$: $n! = n \cdot (n-1) \cdots 1$ (con $0! = 1$). Para $0 \leq r \leq n$:

$$\binom{n}{r} = \frac{n!}{r! (n-r)!}$$

cuenta los subconjuntos de tamaño r de un conjunto de n elementos.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar**
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

El principio

Principio del palomar

Si n objetos se distribuyen en k cajas y $n > k$, entonces **al menos una caja contiene más de un objeto.**

Versión general

Si n objetos van a k cajas, alguna caja contiene al menos $\lceil n/k \rceil$ objetos.

Carácter no constructivo

El principio *garantiza la existencia* del conflicto, pero *no dice* dónde ocurre. Es un argumento de conteo disfrazado.

Ejemplos que conectan con el resto del capítulo

Ejemplo 1 — divisibilidad

Entre $\{1, 2, \dots, n + 1\}$ hay dos que dejan el mismo residuo al dividir por n : hay $n + 1$ números y solo n residuos posibles.

Ejemplo 2 — funciones no inyectivas

Toda función $f : \{1, \dots, n + 1\} \rightarrow \{1, \dots, n\}$ no es inyectiva. Es exactamente la versión “funciones” del principio del palomar.

Ejemplo 3 — cumpleaños

Con 367 personas hay dos que cumplen años el mismo día; con sólo 23, la probabilidad ya supera el 50 %.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas**
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Definición

Dadas $g : A \rightarrow B$ y $f : B \rightarrow C$, la **composición** $f \circ g : A \rightarrow C$ se define por

$$(f \circ g)(a) = f(g(a)), \quad a \in A.$$

- **No es conmutativa** en general: $f \circ g \neq g \circ f$.
- **Identidad**: $f \circ \text{id}_A = f = \text{id}_B \circ f$.
- **Asociatividad**: $(f \circ g) \circ h = f \circ (g \circ h)$.
- Componer inyectivas (resp. sobreyectivas) da inyectiva (resp. sobreyectiva).

Función inversa: deshacer una función

Definición

Si $f : A \rightarrow B$ es **biyectiva**, su **inversa** $f^{-1} : B \rightarrow A$ se define por

$$f^{-1}(b) = a \iff f(a) = b.$$

- $f \circ f^{-1} = \text{id}_B$ y $f^{-1} \circ f = \text{id}_A$.
- f tiene inversa $\iff f$ es biyectiva.
- Si f y g son biyectivas, $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$ (el orden se *invierte*).

Idea

Las biyecciones son las funciones **reversibles**. Por eso la biyectividad es tan importante en matemática y en criptografía.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional**
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

¿Qué pregunta la complejidad computacional?

Idea

La **complejidad computacional** clasifica los problemas según los *recursos* (tiempo, memoria) que necesita cualquier algoritmo que los resuelva, en función del tamaño n de la entrada.

Vocabulario esencial

- **Problema:** pregunta genérica a resolver.
- **Instancia:** un caso particular con datos concretos.
- **Algoritmo:** procedimiento paso a paso que resuelve el problema.
- **Recurso:** tiempo (número de operaciones) o espacio (memoria).

Definición

Decimos $f(n) = O(g(n))$ si existen constantes $C, n_0 > 0$ tales que

$$|f(n)| \leq C \cdot g(n) \quad \text{para todo } n \geq n_0.$$

- $O(\cdot)$ describe el **comportamiento asintótico**: ignora constantes y términos de menor orden.
- Útil porque dos algoritmos con constantes distintas pero misma $O(\cdot)$ escalan igual cuando n crece.

Orden, de menor a mayor

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n.$$

- **Tiempo polinomial:** $O(n^k)$ para alguna k (clase **P**).
- **Tiempo exponencial:** $O(c^n)$ con $c > 1$.
- La frontera polinomial/exponencial *no* es sólo un tecnicismo: marca la diferencia entre **resoluble** y **intratable** en la práctica.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos**
- 11 Resumen y repaso histórico

Definición

El **análisis de algoritmos** determina, asintóticamente, el número de operaciones elementales que un algoritmo realiza, en función del tamaño n de la entrada.

Tres medidas estándar

- **Peor caso:** $T_{\max}(n) = \max_{|x|=n}(\text{ops}(x))$. Cota superior universal.
- **Caso promedio:** $T_{\text{avg}}(n) = \mathbb{E}[\text{ops}(X)]$ sobre una distribución de entradas.
- **Mejor caso:** $T_{\min}(n) = \min_{|x|=n}(\text{ops}(x))$. Útil sólo como información complementaria.

Ejemplo: búsqueda lineal

Problema

Buscar un elemento x en un arreglo $A[1..n]$ recorriendo las posiciones en orden y devolviendo “no está” si no aparece.

- **Mejor caso:** $x = A[1]$, $T_{\min}(n) = O(1)$.
- **Peor caso:** $x \notin A$ o $x = A[n]$, $T_{\max}(n) = O(n)$.
- **Caso promedio:** x en posición uniforme, $T_{\text{avg}}(n) = O(n)$.

Conclusión

La búsqueda lineal es $O(n)$ en el peor caso. Un análisis cuidadoso *distingue* mejor / peor / promedio, no los mezcla.

Las herramientas que aprendimos en el capítulo se conectan aquí:

- Las **funciones** aparecen al describir algoritmos ($f : \text{Entrada} \rightarrow \text{Salida}$).
- El **principio del palomar** explica por qué ciertos problemas *deben* ser lentos.
- **Composición** y **recursión** justifican las recurrencias que aparecen al analizar algoritmos divide-y-vencerás.
- La **notación** O resume el comportamiento de esas recurrencias.

Contenido

- 1 Motivación: por qué necesitamos relaciones y funciones
- 2 Productos cartesianos y relaciones
- 3 Funciones: definición general
- 4 Funciones inyectivas y sobreyectivas
- 5 Conteo de funciones sobreyectivas: Stirling
- 6 Funciones especiales
- 7 El principio del palomar
- 8 Composición de funciones e inversas
- 9 Complejidad computacional
- 10 Análisis de algoritmos
- 11 Resumen y repaso histórico

Resumen del capítulo: el hilo conductor

- ➊ **Producto cartesiano y relación:** el lenguaje para describir interacciones entre conjuntos.
- ➋ **Función:** la relación sin ambigüedad; inyectiva, sobreyectiva, biyectiva.
- ➌ **Conteo:** funciones sobreyectivas $= k! S(n, k)$ vía particiones.
- ➍ **Funciones auxiliares:** constante, identidad, piso, techo, característica, factorial.
- ➎ **Principio del palomar:** herramienta de existencia no constructiva.
- ➏ **Composición e inversa:** cómo combinar y revertir funciones.
- ➐ **Complejidad y análisis:** traducir problemas a funciones y medir su costo asintótico.

- **Georg Cantor** (s. XIX): fundador de la teoría de conjuntos; los conjuntos son el lenguaje base del capítulo.
- **James Stirling** (s. XVIII): los números que llevan su nombre, herramientas centrales en combinatoria.
- **Peter Gustav Lejeune Dirichlet** (s. XIX): formulaciones rigurosas del principio del palomar (*cajón de Dirichlet*).
- **Alonzo Church** y **Alan Turing** (s. XX): formalizan la noción de “función computable” y, con ella, la idea misma de *algoritmo*.
- **Donald Knuth** (s. XX): “The Art of Computer Programming”; formaliza el análisis asintótico de algoritmos.